



Tema 5 - Microarquitectura - Pipeline Escalar En RISC

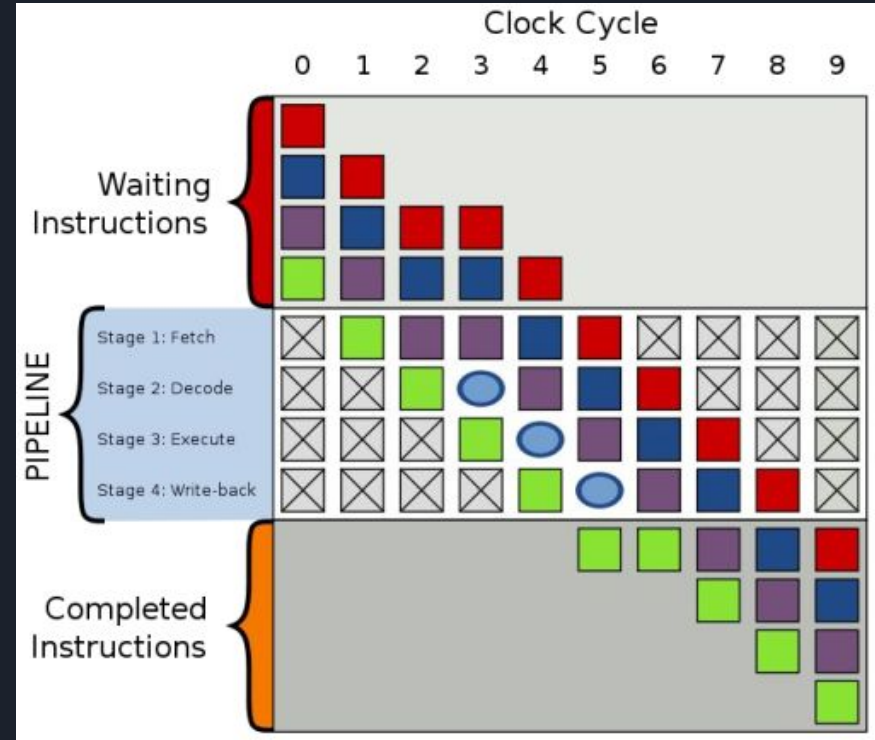
Cátedra: Organización del Computador II
Lic. Juan Pablo Moreno

Línea de montaje



¿Que es ?

- Utilizar un pipeline consiste en segmentar circuitos
- En vez de tener un gran circuito que ejecuta una sola instrucción, separo en UF.
- Aprovecho circuitos que quedarían en desuso.
- Cada Unidad Funcional tiene una tarea específica
 - Pierdo en latencia
 - Luego de los primeras n etapas gano en paralelismo





Pipeline

- Introduce paralelismo a nivel de instrucción permitiendo la ejecución solapada de las instrucciones
- Análogo a una línea de montaje
- Ciclo de instrucción:
 - Buscar
 - Decodificar
 - Ejecución
 - Almacenamiento del resultado
- Cuantas más etapas tenga el pipeline mayor será el trabajo realizado (Throughput)
- $\text{Tiempo por instrucción (promedio)} = \text{Tiempo sin pipeline} / \text{Cantidad de etapas}$



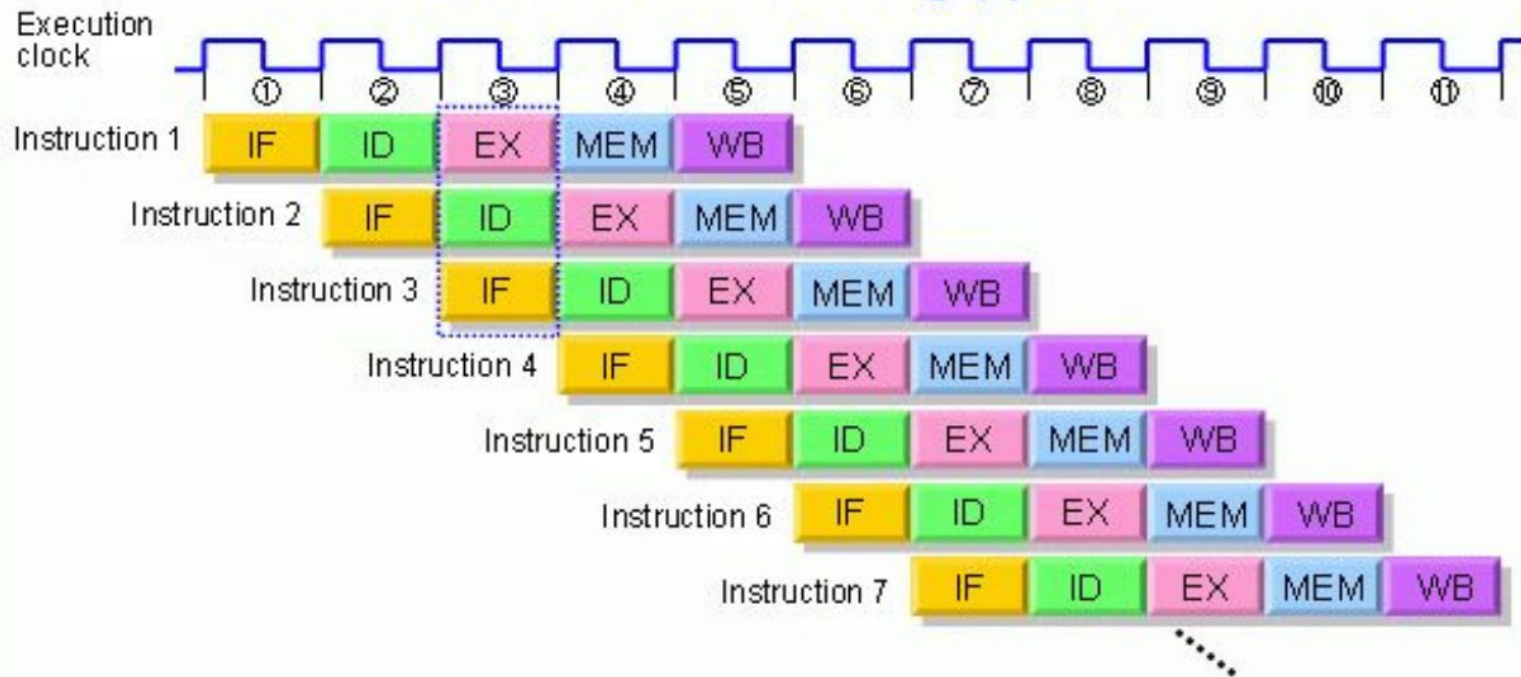
Ciclos de ejecución de una instrucción (RISC)

Tomaremos como ejemplo el ciclo de instrucciones en una arquitectura RISC

1. Búsqueda de Instrucción (IF)
2. Decodificación / Búsqueda de registro (ID)
3. Ejecución / Dirección Efectiva: Cálculo de dirección, Operación ALU reg reg, u Operación ALU reg inmediato (EX)
4. Acceso a Memoria (Load o Store) (MA)
5. Escritura (Write Back) (WB)

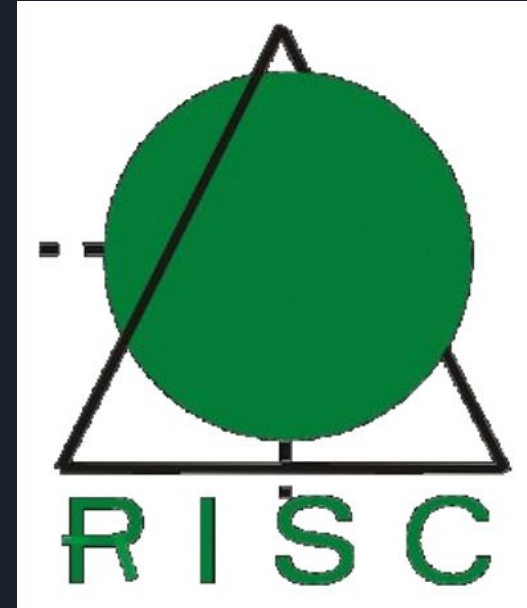
RISC y el pipeline de cinco etapas

Instruction execution in 5-stage pipeline



Instrucciones RISC

- Arithmetic & logic: IF, ID, EX, WB
 - DSUB R1, R2, R3
 - DADD R1, R2, R3
- Load: IF, ID, EX, MA, WB
 - LD R1, offset(R2)
- Store: IF, ID, EX, MA
 - SD R1, offset(R2)
- Branch: IF, ID, EX
 - BEQ R1, R2, offset(R2) *BEQ= Branch Equals
- Y MOV?
 - No tenemos...



Instrucciones RISC

Tipo de instrucción	IF	ID	EX	MA	WB
DADD / DSUB	✓	✓	✓	-	✓
LD	✓	✓	✓	✓	✓
SD	✓	✓	✓	✓	-
BEQ	✓	✓	✓	-	-

Riesgos del pipeline (Hazards)

- Situaciones que impiden la ejecución de una etapa en el instante de tiempo designado
- Riesgos estructurales: Conflictos de recursos. El hardware no soporta simultáneamente todos los tipos de instrucciones.
- Riesgos de datos: Una instrucción depende del resultado de otra instrucción previa.
- Riesgos de control: Ejecución en el pipeline de instrucciones que cambian el flujo del programa (IP)





Riesgos Estructurales

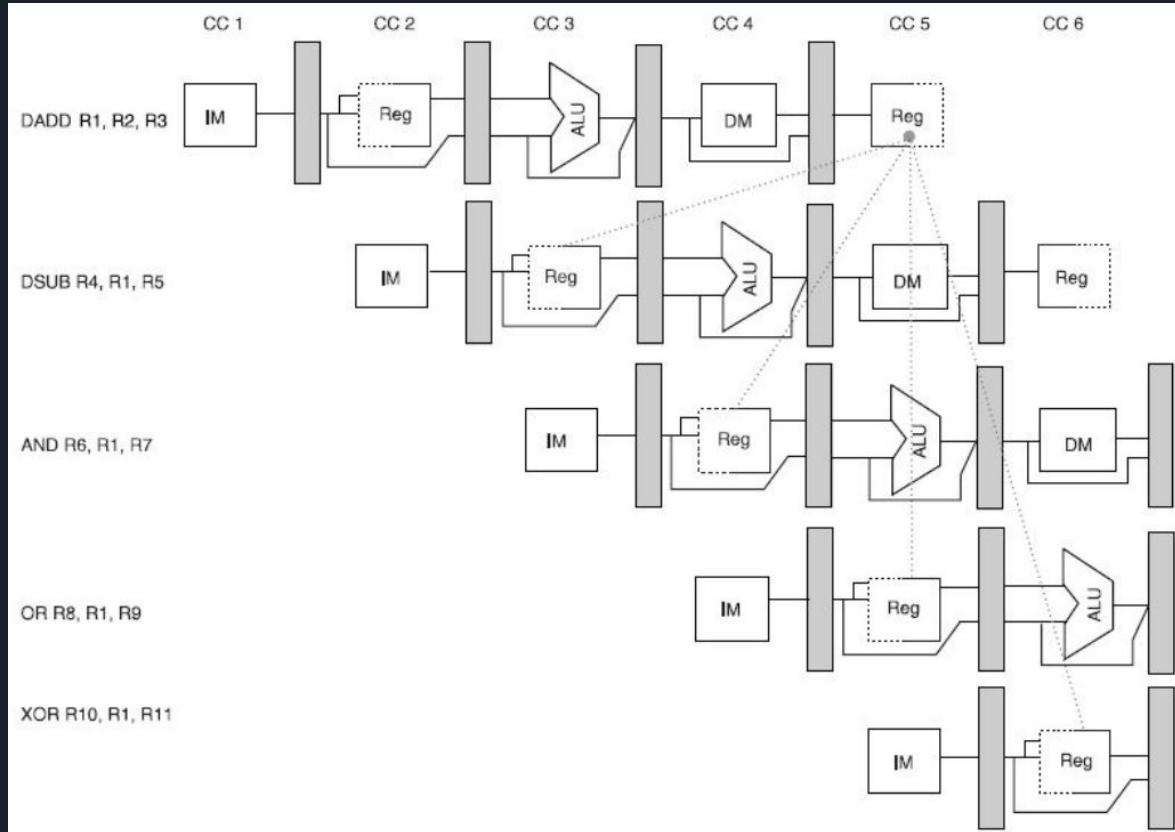
- Conflictos de recursos
 - Pueden surgir con unidades funcionales que no están completamente adaptadas al pipeline
 - Recursos que no se duplican lo suficiente para permitir todas las combinaciones de instrucciones
- Resultado
 - La instrucción se detiene (Stall) hasta que la unidad requerida esté disponible
 - El stall se propaga a las demás instrucciones (burbuja)
- Solución
 - Duplicar Recursos... veremos más adelante con Pipelines SuperEscalares

Riesgos de datos

Los riesgos de datos ocurren cuando el pipeline cambia el orden de las lecturas y escrituras.

DADD	R1,R2,R3
DSUB	R4,R1,R5
AND	R6,R1,R7
OR	R8,R1,R9
XOR	R10,R1,R11

Riesgos de datos

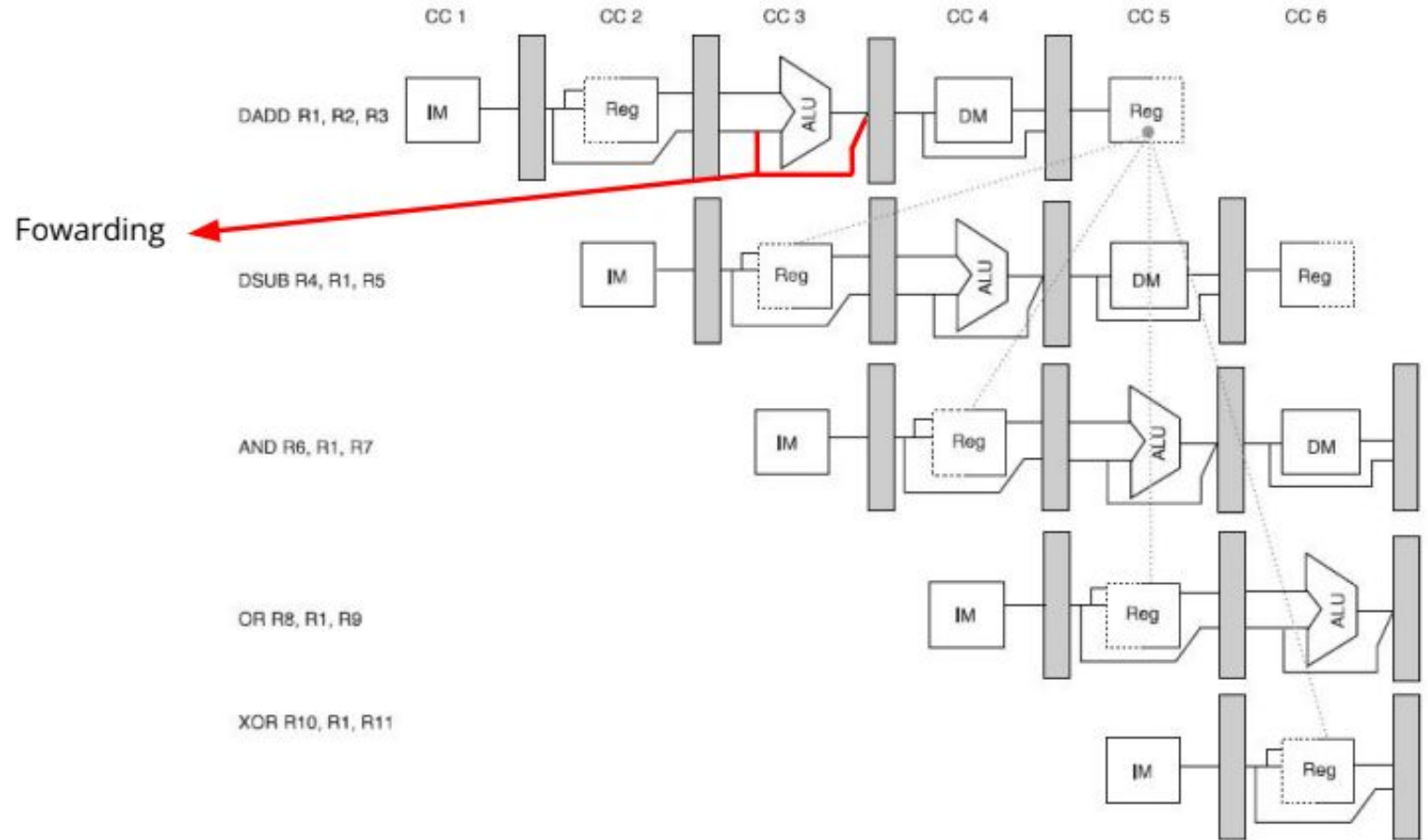




Riesgo de datos, Solución: Forwarding

El resultado de la primer operación está disponible una vez que opera la ALU

- No hace falta que el resultado sea escrito para que las demás instrucciones lo usen.
- El resultado de la ALU alimenta las entradas de la ALU
- Si la lógica de control detecta que el destino de la operación anterior es una entrada de la siguiente, usa el valor adelantado en vez del leído de los registros.



Riesgos de Control

- Los riesgos de control pueden causar una pérdida mayor en la performance
- Al ejecutar una bifurcación puede modificarse o no el puntero a la próxima instrucción.
- Si no se toma la bifurcación, el registro IP no cambia.
- Si se toma la bifurcación la instrucción siguiente debe volver a buscarse.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

Riesgos de control, Solución: Predicción de Saltos

- Detener el pipeline hasta que se conoce el destino del salto (más simple, pero incurre siempre en penalidad)
- Tomar todos los saltos como "no tomado" (Predicted-not-taken)
 - si el salto es tomado hay que deshacer y buscar la instrucción correcta
- Tomar todos los saltos como "tomado" (Predicted taken)
 - Hay que deshacer igual
- Los compiladores pueden generar código que aprovecha el esquema de predicción de saltos.
 - Sabiendo que va a predecir el CPU optimizamos con eso en mente para que en su predicción cometa menos errores.

Loop unrolling - Planificación Estática

- Replicar el cuerpo del ciclo para permitir el solapamiento de las instrucciones
- Las iteraciones deben ser independientes
- Usar registros distintos para no agregar dependencias
- Elimina comparaciones y bifurcaciones
- Preservar dependencias para que el resultado sea consistente
- Planificación estática (compilador)

```
for (i = 1000; i!=0 ; i--){  
    a[i]=a[i]+5  
}
```



```
for( i = 0 ; i<=1000 ; i+=50 ){  
    a[i]=a[i]+5  
    a[i+2]=a[i+2]+5  
    .  
    .  
    .  
    a[i+50]=a[i+50]+5
```



Pipeline

- Desventajas:
 - La implementación del pipeline introduce un overhead
 - Riesgos u obstáculos del Pipeline
 - Los riesgos pueden frenar (Stall) la ejecución de una etapa del pipeline.
 - Las instrucciones emitidas después que la instrucción detenida también se detienen.
 - Las instrucciones emitidas antes, continúan su ejecución.
- Ventajas:
 - En promedio el tiempo de ejecución promedio de las instrucciones se reduce.

Gracias!

